

Exercícios Complementares — Comunicação Interprocessos

1. Considere o servidor web *multithreaded* usado como exemplo na aula de *threads*. O servidor possui uma *thread* despachante, que recebe requisições pela rede e armazena cada requisição em uma fila, e um conjunto de *threads* operárias, que recuperam a próxima requisição da fila e efetuam o seu processamento. O pseudocódigo está mostrado abaixo; a *thread* despachante executa a função `despachante()`, e as *threads* operárias executam a função `operaria()`. Para evitar problemas de concorrência, é necessário garantir a **exclusão mútua** entre as *threads* no acesso à fila de requisições. Mostre como isso poderia ser implementado usando um ou mais semáforos. Não se preocupe em eliminar a espera ocupada do laço das linhas 12–13.

```
1 void despachante() {
2     buf_t buf;
3     while (TRUE) {
4         get_next_request(&buf); // le próxima requisição, armazena em buf
5         handoff_work(&buf);      // insere buf na fila de requisições
6     }
7 }
8
9 void operaria(int i) {
10    buf_t buf;
11    while (TRUE) {
12        while (empty(req_queue))           // se a fila estiver vazia,
13            sleep(1);                       // dá chance para despachante
14        get_request(&buf);                  // pega requisição da fila
15        look_for_page_in_cache(&buf, &page); // procura página no cache
16        if (page_not_in_cache(&page))      // se não encontrar,
17            read_page_from_disk(&buf, &page); // lê página do disco
18        return_page(&page);                // retorna página para usuário
19    }
20 }
```

2. Suponha que o código do servidor web do exercício anterior foi refatorado. Agora, cada *thread* operária insere a página a ser enviada para o cliente em uma fila de respostas, e existe uma única *thread* que processa essa fila e envia as respostas via rede. O pseudocódigo das *threads* operárias e da *thread* respondente é mostrado abaixo; observe que a *thread* despachante não sofreu alteração. Mostre como a solução do exercício anterior, baseada em semáforo(s), poderia ser adaptada para implementar exclusão mútua nesta nova versão.

```
1 void despachante() {
2     buf_t buf;
3     while (TRUE) {
4         get_next_request(&buf); // le próxima requisição, armazena em buf
5         handoff_work(&buf);     // insere buf na fila de requisições
6     }
7 }
8
9 void operaria(int i) {
10    buf_t buf;
11    page_t page;
12    while (TRUE) {
13        while (empty(req_queue)) // se a fila estiver vazia,
14            sleep(1);           // dá chance para despachante
15        get_request(&buf);      // pega requisição da fila
16        look_for_page_in_cache(&buf, &page); // procura página no cache
17        if (page_not_in_cache(&page)) // se não encontrar...
18            read_page_from_disk(&buf, &page); // lê página do disco
19        insert_page(&page);     // insere página na fila
20    }
21 }
22
23 void respondente(void) {
24    page_t page;
25    while (TRUE) {
26        while (empty(page_queue)) // se a fila estiver vazia,
27            sleep(1);           // dá chance para operária inserir
28        wait_for_page(&page);    // pega página da fila
29        return_page(&page);     // retorna página para usuário
30    }
31 }
```

3. Considere um programa concorrente com três *threads*, X, Y e Z, mostradas abaixo.

int n = 1; /* variável compartilhada entre X, Y e Z */		
void X() {	void Y() {	void Z() {
n = n * 16;	n = n / 7;	n = n + 40;
}	}	}

Mostre como as *threads* poderiam usar semáforos para garantir que o resultado final de n seja sempre 8.

4. No problema do exercício 2, substitua os laços de espera ocupada das linhas 13–14 e 26–27 por uma solução bloqueante usando semáforo(s).

5. O JEC Futsal encomendou um sistema para venda de ingressos para seus jogos através de quiosques eletrônicos. As partidas são realizadas no Centreventos Cau Hansen, que tem capacidade para MAX torcedores; os lugares são numerados sequencialmente, de 1 a MAX. O sistema deve funcionar da seguinte forma: o cliente escolhe o lugar que deseja (dentre os disponíveis) e efetua o pagamento. O pagamento é feito por cartão, e pode ser aprovado pela operadora ou não (neste caso, o cliente perde o direito ao lugar escolhido, que volta a ficar disponível). O sistema não pode vender duas vezes o mesmo lugar, nem pode marcar como ocupados lugares escolhidos por clientes que não conseguem efetuar o pagamento. Apenas um lugar é vendido por vez. Escreva o pseudocódigo das aplicações que executam nos quiosques, usando semáforos para evitar condições de disputa. Considere que a autorização de pagamento é feita pela Internet, estando sujeita a falhas de conexão e a longos tempos de espera (ou seja, um cliente que aguarda a autorização de pagamento não pode prejudicar a venda de ingressos em outros quiosques).